



10. Feature Selection

Less is More – Select the Right Features, Build Better Models.

Next Section:

 [9.5. Evaluation Metrics](#)

Contents:

[10.1. Correlation-Based Selection](#)

[2. Filter Methods](#)

[10.3. Embedded Methods](#)

[10.4. Wrapper Methods](#)

[Summary](#)

[References](#)

In real-world datasets, not every feature is useful for solving a problem. Some features provide valuable information, while others are irrelevant, redundant, or simply add noise. Including unnecessary features can increase training time, reduce model accuracy, and even lead to overfitting.

- For example, suppose we want to predict **house prices**.
 - Features such as the **house area**, **number of bedrooms**, and **location** are likely to be useful.
 - However, attributes like the **owner's favorite TV show** or **their children's hobbies** have no logical relationship with the selling price and only make the dataset more complicated.
- Similarly, when building a **spam email classifier**.

- Features such as the **number of suspicious keywords**, **number of hyperlinks**, or **whether the sender is trusted** can help distinguish spam from legitimate emails.
- On the other hand, information such as the **writer's favorite sport** or the **number of apps installed on their phone** has nothing to do with whether an email is spam.

The goal of **feature selection** is to identify the most informative features while removing those that contribute little or no useful information. This often leads to simpler, faster, and more accurate machine learning models.

Throughout this section, we will create two synthetic datasets with **200 samples**:

- A **House Price** dataset for regression.
- A **Spam Detection** dataset for classification.

These datasets will help us demonstrate several common feature selection techniques.

```
import numpy as np
import pandas as pd

np.random.seed(42)
n = 200
```

- House Price Dataset

```
house_df = pd.DataFrame({
    "Area": np.random.randint(60, 250, n),
    "Bedrooms": np.random.randint(1, 6, n),
    "Age": np.random.randint(1, 35, n),
    "Distance_to_City": np.random.randint(1, 25, n),

    # Irrelevant features
    "Favorite_TV_Show": np.random.choice(
        ["Friends", "Naruto", "Football", "News", "Drama"],
        n
    ),
})
```

```

        "Children_Hobbies": np.random.choice(
            ["Music", "Gaming", "Reading", "Swimming"], n
        )
    })

house_df["Price"] = (
    house_df["Area"] * 2800
    + house_df["Bedrooms"] * 18000
    - house_df["Age"] * 1200
    - house_df["Distance_to_City"] * 4000
    + np.random.normal(0, 25000, n)
).astype(int)

print(house_df.head(20))

```

- Spam Detection Dataset

```

spam_df = pd.DataFrame({
    "Spam_Words": np.random.randint(0, 25, n),
    "Num_Links": np.random.randint(0, 12, n),
    "Has_Attachment": np.random.randint(0, 2, n),

    # Irrelevant features
    "Favorite_Sport": np.random.choice(
        ["Football", "Basketball", "Tennis", "Chess"], n
    ),
    "Phone_Apps": np.random.randint(10, 150, n)
})

score = (
    spam_df["Spam_Words"] * 2
    + spam_df["Num_Links"] * 3
    + spam_df["Has_Attachment"] * 2
)

spam_df["Spam"] = (score > 24).astype(int)

```

```
print(spam_df.head(20))
```

10.1. Correlation-Based Selection

One of the simplest feature selection techniques is **Correlation-Based Selection**. In the EDA section, we learned that a **correlation matrix** measures the strength of the linear relationship between numerical variables.

The basic idea is straightforward:

- Features with **strong correlation** to the target variable are usually more useful.
- Features with **very weak or near-zero correlation** often contribute little information and may be removed.
- If two features are highly correlated with each other, keeping both may be unnecessary because they provide similar information.

Correlation-based selection is easy to compute, fast, and often serves as the first step before applying more advanced feature selection methods. However, it only measures **linear relationships** and cannot detect more complex nonlinear patterns.

Example: House Price Dataset

- First, remove the categorical columns since correlation only applies to numerical features.

```
house_numeric = house_df.drop(
    columns=["Favorite_TV_Show", "Children_Hobbies"]
)

house_numeric.corr().round(2)
```

- To see which features are most related to the target variable:

```
house_numeric.corr()["Price"].sort_values(ascending=False)
```

- Example output:

Feature	Correlation with Price
Price	1.00
Area	0.957
Bedrooms	0.1194
Age	0.03
Distance_to_City	-0.13

From the correlation values, we can conclude that **Area** has the strongest relationship with house price, followed by **Bedrooms**, while **Age** and **Distance_to_City** have weaker negative relationships.

Since the two categorical columns are irrelevant to the prediction task, they are excluded before the analysis.

2. Filter Methods

Filter methods select features by applying statistical measures **before** training a machine learning model. Instead of evaluating different models, these methods rank each feature independently according to its relationship with the target variable.

Common filter techniques include:

- Correlation coefficient (regression)
- Chi-Square test (classification)
- ANOVA F-test
- Mutual Information

Because filter methods do not require repeated model training, they are **fast**, **computationally efficient**, and suitable for high-dimensional datasets.

However, they evaluate each feature individually and therefore may miss interactions between multiple features.

One of the most widely used implementations is **SelectKBest**, which ranks features according to a chosen statistical score and keeps only the top **k** features.

Example: Selecting the Top 3 Features

```
from sklearn.feature_selection import SelectKBest, f_regression

X = house_numeric.drop(columns="Price")
y = house_numeric["Price"]

selector = SelectKBest(score_func=f_regression, k=3)

X_selected = selector.fit_transform(X, y)

selected_features = X.columns[selector.get_support()]

print(selected_features)
```

Example output:

```
Index(['Area', 'Bedrooms', 'Distance_to_City'], dtype='object')
```

- To display the statistical scores:

```
scores = pd.DataFrame({
    "Feature": X.columns,
    "Score": selector.scores_
})

scores.sort_values("Score", ascending=False)
```

Example output:

Feature	Score
Area	2186.73
Bedrooms	3.58
Distance_to_City	2.8645
Age	0.282

The larger the score, the more informative the feature is for predicting the target variable. Based on these scores, **SelectKBest** automatically retains the three highest-ranked features while discarding the remaining ones.

10.3. Embedded Methods

Unlike filter methods, **embedded methods** perform feature selection **during the model training process**. As the model learns, it automatically determines which features contribute the most to making accurate predictions.

Many tree-based models naturally calculate **feature importance** by measuring how much each feature helps reduce prediction error when splitting the data.

Likewise, regularized linear models such as **Lasso Regression (L1 Regularization)** shrink the coefficients of less useful features toward zero, effectively removing them.

Embedded methods generally produce better feature subsets than filter methods because they consider the learning algorithm itself. However, the selected features depend on the chosen model.

Some common embedded methods include:

- Decision Trees
- Random Forest
- Gradient Boosting
- XGBoost
- Lasso Regression

Example: Random Forest Feature Importance

```
from sklearn.ensemble import RandomForestRegressor

X = house_numeric.drop(columns="Price")
y = house_numeric["Price"]

model = RandomForestRegressor(
    n_estimators=100,
    random_state=42
```

```

)

model.fit(X, y)

importance = pd.DataFrame({
    "Feature": X.columns,
    "Importance": model.feature_importances_
})

importance.sort_values(
    by="Importance",
    ascending=False
)

```

Example output

Feature	Importance
Area	0.93
Distance_to_City	0.034
Bedrooms	0.018
Age	0.009

The importance values indicate how much each feature contributes to the model's predictions. Features with larger importance values have a greater influence on the predicted house price, while features with very small values may be considered for removal.

10.4. Wrapper Methods

Wrapper methods are among the most powerful feature selection techniques. Instead of evaluating each feature individually, they repeatedly train a machine learning model using different combinations of features and select the subset that produces the best performance.

Because wrapper methods evaluate feature subsets directly, they usually achieve higher predictive performance than filter methods. However, this comes at the cost of significantly higher computational time.

The three most common wrapper methods are:

- **Forward Selection** – Starts with no features and adds one feature at a time.
- **Backward Elimination** – Starts with all features and removes the least useful feature at each step.
- **Recursive Feature Elimination (RFE)** – Recursively removes the least important feature until the desired number of features remains.

Among these methods, **Recursive Feature Elimination (RFE)** is the most widely used because it provides a good balance between accuracy and computational efficiency.

Suppose we have a house price dataset with five features:

- Number of Bedrooms
- House Area
- House Color
- Number of Family Members
- Number of Neighboring Houses

Although all five features are available, not all of them are equally useful. RFE helps identify which subset produces the best predictive performance.

Creating an Example Dataset

```
np.random.seed(42)

n = 200

wrapper_df = pd.DataFrame({
    "Bedrooms": np.random.randint(1, 6, n),
    "Area": np.random.randint(70, 250, n),
    "House_Color": np.random.randint(0, 5, n),
    "Family_Members": np.random.randint(1, 8, n),
    "Neighbor_Houses": np.random.randint(2, 20, n)
})

wrapper_df["Price"] = (
    wrapper_df["Area"] * 2800
    + wrapper_df["Bedrooms"] * 20000
```

```

+ wrapper_df["Family_Members"] * 3000
+ np.random.normal(0, 20000, n)
)

print(wrapper_df.head(20))

```

- Recursive Feature Elimination (RFE)

```

from sklearn.feature_selection import RFE
from sklearn.linear_model import LinearRegression

X = wrapper_df.drop(columns="Price")
y = wrapper_df["Price"]

model = LinearRegression()

selector = RFE(
    estimator=model,
    n_features_to_select=3
)

selector.fit(X, y)

selected_features = X.columns[selector.support_]

print(selected_features)

```

Example output: Index(['Bedrooms', 'Area', 'Family_Members'], dtype='object')

- To see the ranking of every feature:

```

ranking = pd.DataFrame({
    "Feature": X.columns,
    "Ranking": selector.ranking_
})

```

```
ranking.sort_values("Ranking")
```

Example output

Feature	Ranking
Bedrooms	1
Area	1
Family_Members	1
Neighbor_Houses	2
House_Color	3

A ranking of **1** indicates that the feature was selected by RFE. Higher ranking values indicate features that were eliminated earlier because they contributed less to the model's performance.

Summary

In this section, we introduced four of the most widely used feature selection techniques for preparing data before building machine learning models.

- **Correlation-Based Selection** uses correlation analysis to identify features that are strongly related to the target variable.
- **Filter Methods** rank features using statistical measures before any model is trained, making them fast and efficient for large datasets.
- **Embedded Methods** perform feature selection automatically during model training, with tree-based and regularized models providing feature importance scores.
- **Wrapper Methods** evaluate different combinations of features using repeated model training. Although computationally expensive, they often produce the most effective feature subsets.

Selecting relevant features reduces model complexity, speeds up training, improves interpretability, and can significantly increase predictive performance by eliminating irrelevant or redundant information.

References

https://scikit-learn.org/stable/modules/feature_selection.html

<https://www.geeksforgeeks.org/machine-learning/feature-selection-techniques-in-machine-learning/>

<https://www.ibm.com/think/topics/feature-selection>

Next Section:

 [11. Multiple Imputation by Chained Equation \(MICE\)](#)